

# Deliverable report

“Alessio Blascovich”: Mat: 257561, [alessio.blascovich@studenti.unitn.it](mailto:alessio.blascovich@studenti.unitn.it), GitRepo: <https://github.com/elblasco/GPU-Computing-2025-257561>

**Abstract**—The sparse matrix-dense vector multiplication (SpMV) is an algebraic operation in which a sparse matrix is multiplied by a dense vector. SpMV is widely used in real-world applications such as scientific and engineering computing, data mining, and graph analysis.

This deliverable discusses the implementation and the performance analysis of a series of SpMV algorithms for COO matrices. These algorithms are implemented for a single-core CPU and a CUDA capable GPU. Regarding the GPU implementations, no threads/warp level synchronisation has been used, only atomic operations to keep the resulting vector consistent.

**Index Terms**—Sparse Matrix, SpMV, CUDA, Parallelization, Storage Format

## I. INTRODUCTION

SpMV is considered a fundamental algebraic operation in many scientific fields such as computational fluid dynamics, optimisation and graph algorithms.

The main challenges in implementing an efficient SpMV algorithm differ in CPU and GPU, and are strictly bound to the type of matrix representation used (COO, CSR, ELL,...). For the purpose of this deliverable only COO representation is taken into account.

From the CPU point of view, the main issue is to maximise the cache hits throughout the execution, data consistency is not a problem since only single core CPUs were used.

On the GPU side the problem of maximising cache hits remains, and on top of that, race conditions are added. Since in COO format the length of a row is not specified, multiple threads could be reading different portions of the same row and therefore, writing on the same output memory area.

## II. PROBLEM STATEMENT

The SpMV operation is formally defined in Equation (1), where  $\mathbf{A}$  is a  $n \times m$  sparse matrix,  $\mathbf{X}$  a  $m \times 1$  dense vector and  $\mathbf{Y}$  another  $n \times 1$  dense vector.

$$\mathbf{Y} = \mathbf{A} \cdot \mathbf{X} \quad (1)$$

### A. Storage Format

The COOrdinate format (COO) is a representation for sparse matrices. COO uses 3 different arrays to store all the necessary information. One array is dedicated to store all the values, while the other 2 contain the corresponding row and column indices, ideally the arrays are sorted first by row index and then by columns index. An example of a COO encoding is shown in Figure 1.

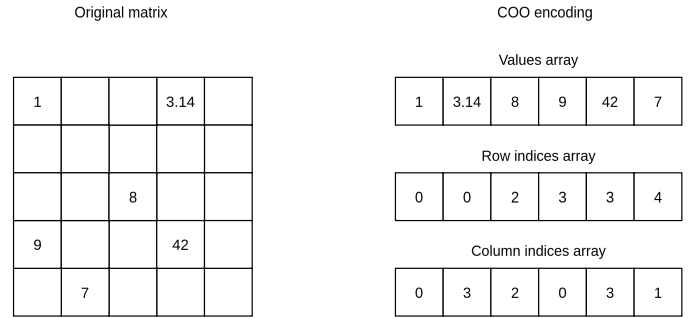


Fig. 1. encoding of a matrix on COO format

### B. Parallelization

Three different kernels have been used for the CUDA capable GPU. Although they differ by the kind of memory and optimisations used, all of them leverage on the fact that the COO arrays are sorted first by row and then by column. In general the multiplication problem has been reduced to a prefix sum computation. In to optimise the computation of prefix I tried to minimise global memory writes by using warp level primitives and the shared memory.

## III. STATE OF THE ART

Starting from the implementations presented in this deliverable, performances could be drastically enhanced. Fukaya et al. [1], despite focussing on the HDC storage format, improved the SpMV CPU based by exploiting diagonalization patterns, SIMD instructions and *OpenMP* directives. On the GPU side Chu et al. [2] used the CSR format and a more sophisticated load balancing to improve the performances. NVIDIA itself made available a set of libraries to manage and use COO formatted matrices, namely *cuSPARSE*.

In the NVIDIA library *cuSPARSE*, and many others commercial libraries, the problem is redefined according to Equation (2).

$$\mathbf{Y} = \alpha op(\mathbf{A}) \cdot \mathbf{X} + \beta \mathbf{Y} \quad (2)$$

With  $\alpha$  and  $\beta$  being two scalars, and  $op$  being a matrix transformation such as a transposition.

## IV. METHODOLOGY AND CONTRIBUTIONS

The test were carried out on a remote cluster. I compared all the kernels against a set of matrices, for every kernel I executed multiple run to collect more meaningful data.

All the algorithms assumed that the resulting array is empty, so I explicitly initialised every element of it to the value 0.

### A. Algorithms

For the GPU implementation I have developed three kernels, they are divided into two main categories. Kernel that leverage on shared memory and kernels that leverage on warp level primitives. Algorithm 1 was the first kernel developed using warp-level primitives. Every thread proceed with a striding of *numThreads* values, initially every thread computes the product between its the matrix value and the value for the corresponding dense array. This values are then combined using a prefix-sum based on warp-level instructions.

---

#### Algorithm 1 SpMV algorithm with WARP level primitives

---

**Require:** The input vectors *val*, *row* and *col* of size *nnz*, array *arr* of size *columns* and an array *res* of size *rows*.

```

1: procedure FUNCTION(row, col, val, arr, res, ...)
2:   start  $\leftarrow$  threadId
3:   pos  $\leftarrow$  threadId & (WARP_SIZE - 1)
4:   total  $\leftarrow$  gridDim · blockDim
5:   for i in {start ... nnz} with i = i + total do
6:     row_id  $\leftarrow$  row[i]
7:     col_id  $\leftarrow$  col[i]
8:     val_id  $\leftarrow$  val[i]
9:     //The last thread per row writes
10:    row_next  $\leftarrow$  shfl_down(0xff..., row, 1)
11:    if Edge_detection() then
12:      atomicAdd(&res[row], prod);
13:    end if
14:  end for
15: end procedure

```

---



---

#### Algorithm 2 Prefix-sum with WARP level primitives

---

**Require:** The values for *product*, *row* and *pos*.

```

1: procedure PREFIX_SUM_WARP(product, row, pos)
2:   for x in {1 ... WARP_SIZE} with x = x · 2 do
3:     prev_row  $\leftarrow$  shfl_up(..., row, x)
4:     prev_prod  $\leftarrow$  shfl_up(..., product, x)
5:     if pos  $\geq$  delta  $\wedge$  prev_row = row then
6:       product = product + rev_prod
7:     end if
8:   end for
9: end procedure

```

---

As mention in Section II-B I reduced the problem to a prefix sum, as a matter of fact Algorithm 2 exploits warp-level primitives to compute a prefix sum. Algorithm 2 is more refined w.r.t. simple prefix sum, since it has to take into account the possibility that within the same warp we could be summing different rows. In Figure 2 it is described a possible execution for the prefix sum procedure, in red the value returned summed in global memory in line 12 of Algorithm 1. The next improvement introduced was the loop unroll of the main loop in Algorithm 2, using compiler directives. To be precise, I used `#pragma unroll`, since

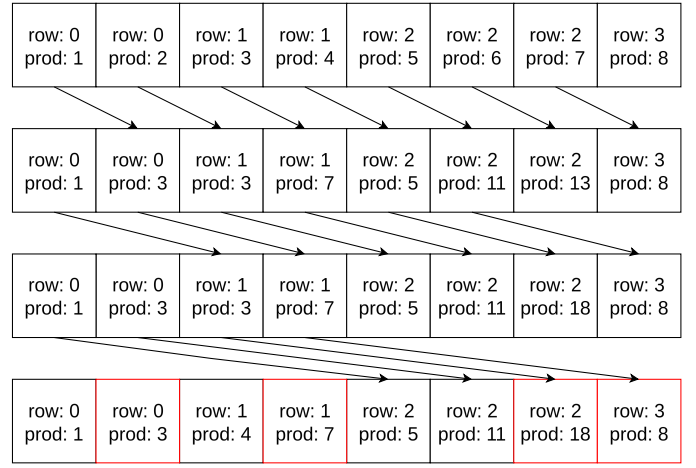


Fig. 2. Execution of prefix sum with warp size 8

the loop always iterates in an interval known at compile time, in fact *WARP\_SIZE* is a macro defined as the value 32.

I also tried to improve the baseline proposed in deliverable 1 by using dynamically allocated shared memory.

## V. SYSTEM DESCRIPTION AND EXPERIMENTAL SET-UP

### A. System Description

For this deliverable I could not use my own machine since it does not have a *CUDA* capable GPU, so I only used the remote cluster. Table I describes all the software used, while Table II describe the hardware that I used.

| Software         | Version | Git Branch | Git Hash |
|------------------|---------|------------|----------|
| GCC              | 11.5.0  |            |          |
| CUDA             | 12.5.0  |            |          |
| SbatchMan        |         | example    | 40d869a  |
| distributed_mmio |         | main       | 3563a2f  |

TABLE I  
SYSTEMS SOFTWARES

| Processor                    | Cores per Socket | RAM    | Accelerator |
|------------------------------|------------------|--------|-------------|
| Intel(R) Xeon(R) Silver 4214 | 48 at 2.2 GHz    | 400 GB | NVidia L40s |

TABLE II  
SYSTEMS DETAILS

### B. Dataset description

To add a bit more of generality to the dataset I picked six matrix from six different kinds from the site <https://sparse.tamu.edu/>. Since all the matrices are squared only the number is written on Table III.

- 1) Electromagnetics Problem;
- 2) Undirected Weighted Graph;
- 3) Optimization Problem;
- 4) Structural Problem;
- 5) 2D/3D Problem;
- 6) Semiconductor Process Problem.

| Name              | Rows        | Pattern Entries | Sparsity Rate |
|-------------------|-------------|-----------------|---------------|
| dielFilterV3real  | 1,102,824   | 89,306,020      | 7e-5          |
| Flan_1565         | 1,564,794   | 117,406,044     | 5e-5          |
| Flan_1565         | 1,564,794   | 117,406,044     | 5e-5          |
| mawi_201512020330 | 226,196,185 | 480,047,894     | 9e-9          |
| nlpkkt160         | 8,345,600   | 229,518,112     | 3e-6          |
| vas_stokes_2M     | 2,146,677   | 65,129,037      | 1e-5          |

TABLE III  
INPUT MATRICES

### C. Experimental Set-up

For every matrix I measured a set of metrics using an approach similar to the first deliverable. For every matrix I spawned a job which ran multiple times every kernel and gathered info about the performances. The metrics measured were: execution time, peak GFLOP/s developed and peak bandwidth used. I was able to test all the algorithms in a single job since I switched to the node *edu02*, which is equipped with the more powerful NVidia L40s, w.r.t. *edu01* which is equipped with NVidia A30.

Every job carried out its measurements performing multiple runs of the same kernel. The kernel tests were split into two period, a warm-up period and a “valid” period, the data has been collected only during the “valid” period. The distinction were necessary in order to war-up the GPU caches.

To automatise the matrix parsing, and their conversion in binary format, I used a fork of `distributed_mmio`<sup>1</sup>.

The job submission were managed by *SbatchMan*, this automation allowed me to create multiple experiments and submit all of them. In particular I created an experiment for normal metrics gathering and an experiment for profiling via *nsys*, *ncu* and *nvtx*.

## VI. EXPERIMENTAL RESULTS

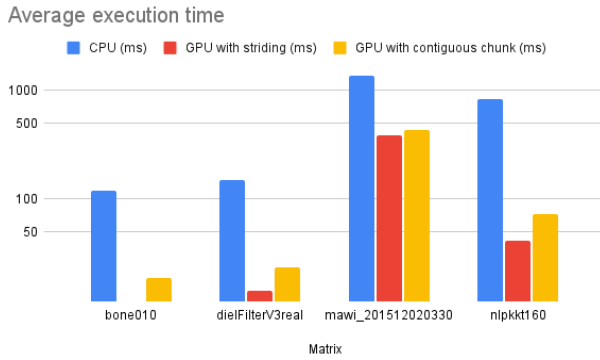


Fig. 3. Average Execution Time (lower is better)

As expected in Section IV and shown in Figure 3 and ??, ?? is faster than all the other implementations.

<sup>1</sup>The fork author is Matteo Di Noia [https://github.com/matteo-dinoia/distributed\\_mmio](https://github.com/matteo-dinoia/distributed_mmio) while I just fixed some minor issues [https://github.com/elblasco/distributed\\_mmio](https://github.com/elblasco/distributed_mmio)

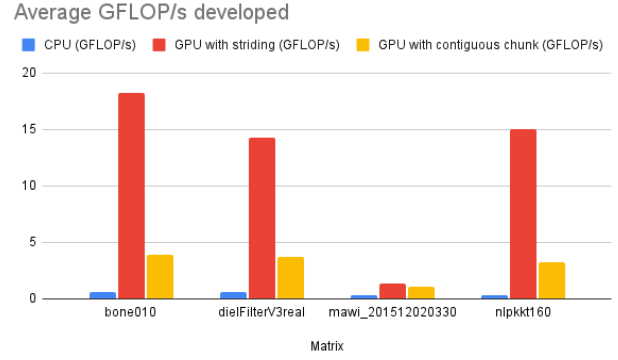


Fig. 4. Average GFLOP/s Developed (lower is better)

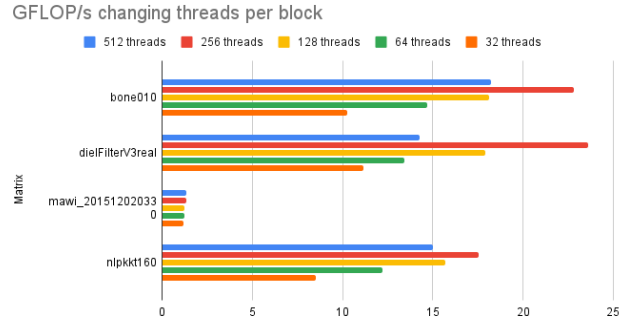


Fig. 5. GFLOP Compared The Number Of Threads Per Block

I decided to dig a bit more into the performances of ?? by changing the maximum amount of thread per block. Starting from the base of 512 threads I divided by 2 until reaching 32, i.e., the size of warp. The optimal amount of threads per block resulted to be 256. This is due to the use of an atomic instruction within the code, the semantic of the instruction make concurrent write over the same address sequential; in this case if two threads are working on the same row their sum operations will be sequentialised since they are trying to write on the same result address. Using less threads, i.e., each thread works on more elements; conflicts (threads working) on the same row are reduced to a minimum. However, this minimum is not stable, in fact decreasing again the value to 128, 64 or 32 the performances decrease again, this could be a symptom of either:

- too few threads working, so an excessive workload for each of them;
- too much threads working on the same row, hence a high number of conflicts on the same row.

Unfortunately, I was not able to determine a more fine grain optimisation, since the optimal size depends on the input matrix shape. Figure 5 and ?? highlights the fact that the variations of performances are not constant over all the matrices considered.

Following the NVIDIA’s white-paper for the A30, I drew the roofline model for single precision floating point, since in

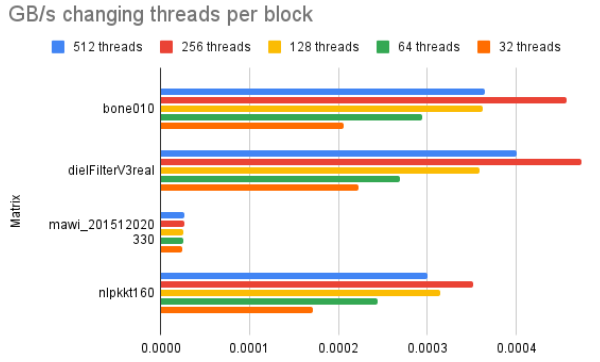


Fig. 6. GB/s Compared The Number Of Threads Per Block

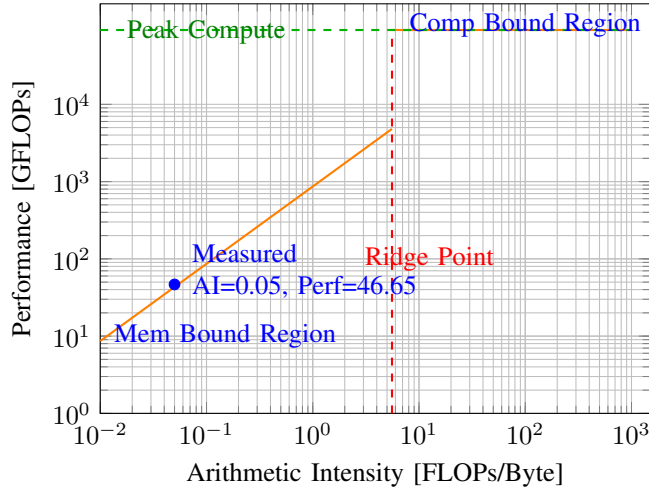


Fig. 7. Roofline Model For SpMV On NVIDIA L40s

my code I used `float` as data type to represent values within the matrix and vectors. Figure 7 highlights the fact that my *CUDA* kernels are indeed a memory bounded.

## VII. CONCLUSIONS

The presented algorithms are far from decent implementations. The representation used for the matrices is sub-optimal since the length of a row is unknown a priori, therefore it is impossible to do a decent load balancing between the threads. For better GPU performances using another representation such as CSR would works better.

Another bottleneck for the algorithm is represented by the use of an atomic instruction. Instead of this instruction thread/warp level synchronisation should be used.

On the CPU side, this representation is optimal since it permits to maximise the number of cache hits working on data stored so closely.

## REFERENCES

- [1] T. Fukaya, K. Ishida, A. Miura, T. Iwashita, and H. Nakashima, "Accelerating the spmv kernel on standard cpus by exploiting the partially diagonal structures," 2021. [Online]. Available: <https://arxiv.org/abs/2105.04937>

- [2] G. Chu, Y. He, L. Dong, Z. Ding, D. Chen, H. Bai, X. Wang, and C. Hu, "Efficient algorithm design of optimizing spmv on gpu," in *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing*, ser. HPDC '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 115–128. [Online]. Available: <https://doi.org/10.1145/3588195.3593002>